

AFRILANG Stdlib

std/c/graphcycle

Français. Bibliothèque AFRILANG 'std/c/graphcycle' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : detectCycleDirected, detectCycleUndirected, cycleEdgeCount, isAcyclic, selfLoopCount, removeSelfLoops, hasParallelEdges.

```
'import "std/c/graphcycle"' · 'use graphcycle'
- 'detectCycleDirected(adj list number, n number) → bool' - 'detect-
CycleUndirected(adj list number, n number) → bool' - 'cycleEdge-
Count(adj list number, n number) → number' - 'isAcyclic(adj list num-
ber, n number) → bool' - 'selfLoopCount(adj list number, n number) →
number' - 'removeSelfLoops(adj list number, n number) → l
```

English.

AFRILANG library 'std/c/graphcycle' (tier complex, category complex). Exports 7 function(s): detectCycleDirected, detectCycleUndirected, cycleEdgeCount, isAcyclic, selfLoopCount, removeSelfLoops, hasParallelEdges.

```
'import "std/c/graphcycle"' · 'use graphcycle'
- 'detectCycleDirected(adj list number, n number) → bool' - 'detect-
CycleUndirected(adj list number, n number) → bool' - 'cycleEdge-
Count(adj list number, n number) → number' - 'isAcyclic(adj list num-
ber, n number) → bool' - 'selfLoopCount(adj list number, n number) →
number' - 'removeSelfLoops(adj list number, n number) → list number'
-
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/graphcycle' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : detectCycleDirected, detectCycleUndirected, cycleEdgeCount, isAcyclic, selfLoopCount, removeSelfLoops, hasParallelEdges.

'import "std/c/graphcycle"' · 'use graphcycle'

```
- `detectCycleDirected(adj list number, n number) → bool`
- `detectCycleUndirected(adj list number, n number) → bool`
- `cycleEdgeCount(adj list number, n number) → number`
- `isAcyclic(adj list number, n number) → bool`
- `selfLoopCount(adj list number, n number) → number`
- `removeSelfLoops(adj list number, n number) → list number`
- `hasParallelEdges(adj list number, n number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphcycle"
use graphcycle
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- detectCycleDirected(adj list number, n number) → bool
- detectCycleUndirected(adj list number, n number) → bool
- cycleEdgeCount(adj list number, n number) → number
- isAcyclic(adj list number, n number) → bool
- selfLoopCount(adj list number, n number) → number
- removeSelfLoops(adj list number, n number) → list
- hasParallelEdges(adj list number, n number) → bool

4. Exemples d'utilisation

```
import "std/c/graphcycle"
use graphcycle
```

```
create result = detectCycleDirected(/* args */)
say result
```

```
create result = detectCycleUndirected(/* args */)
say result
```

```
create result = cycleEdgeCount(/* args */)
say result
```

```
create result = isAcyclic(/* args */)
say result
```

```
create result = selfLoopCount(/* args */)
say result
```

```
create result = removeSelfLoops(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/graphcycle"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module graphcycle

```
function detectCycleDirected(adj list number, n number) returns bool
end
```

```
function detectCycleUndirected(adj list number, n number) returns bool
end
```

```
function cycleEdgeCount(adj list number, n number) returns number
end
```

```
function isAcyclic(adj list number, n number) returns bool
end
```

```
function selfLoopCount(adj list number, n number) returns number
end
```

```
function removeSelfLoops(adj list number, n number) returns list number
end
```

```
function hasParallelEdges(adj list number, n number) returns bool
end
end
```

English

1. Overview

AFRILANG library 'std/c/graphcycle' (tier complex, category complex). Exports 7 function(s): detectCycleDirected, detectCycleUndirected, cycleEdgeCount, isAcyclic, selfLoopCount, removeSelfLoops, hasParallelEdges.

'import "std/c/graphcycle"' · 'use graphcycle'

```
- `detectCycleDirected(adj list number, n number) → bool`
- `detectCycleUndirected(adj list number, n number) → bool`
- `cycleEdgeCount(adj list number, n number) → number`
- `isAcyclic(adj list number, n number) → bool`
- `selfLoopCount(adj list number, n number) → number`
- `removeSelfLoops(adj list number, n number) → list number`
- `hasParallelEdges(adj list number, n number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphcycle"
use graphcycle
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- detectCycleDirected(adj list number, n number) → bool•
- detectCycleUndirected(adj list number, n number) → bool•
- cycleEdgeCount(adj list number, n number) → number•
- isAcyclic(adj list number, n number) → bool•
- selfLoopCount(adj list number, n number) → number•
- removeSelfLoops(adj list number, n number) → list•
- hasParallelEdges(adj list number, n number) → bool

4. Usage examples

```
import "std/c/graphcycle"
use graphcycle
```

```
create result = detectCycleDirected(/* args */)
say result
```

```
create result = detectCycleUndirected(/* args */)
say result
```

```
create result = cycleEdgeCount(/* args */)
say result
```

```
create result = isAcyclic(/* args */)
say result
```

```
create result = selfLoopCount(/* args */)
say result
```

```
create result = removeSelfLoops(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/graphcycle"`` at the top of your file.
- Run ``afriLang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module graphcycle

```
function detectCycleDirected(adj list number, n number) returns bool
end
```

```
function detectCycleUndirected(adj list number, n number) returns bool
end
```

```
function cycleEdgeCount(adj list number, n number) returns number
end
```

```
function isAcyclic(adj list number, n number) returns bool
end
```

```
function selfLoopCount(adj list number, n number) returns number
end
```

```
function removeSelfLoops(adj list number, n number) returns list number
end
```

```
function hasParallelEdges(adj list number, n number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/graphcycle"`
- Vérification : `afriLang check`
- Documentation : <https://afriLang.dev/stdlib/std/c/graphcycle/>

Source (excerpt)

```
module graphcycle

function detectCycleDirected(adj list number, n number) returns bool
end

function detectCycleUndirected(adj list number, n number) returns bool
end

function cycleEdgeCount(adj list number, n number) returns number
end

function isAcyclic(adj list number, n number) returns bool
end

function selfLoopCount(adj list number, n number) returns number
end

function removeSelfLoops(adj list number, n number) returns list number
end

function hasParallelEdges(adj list number, n number) returns bool
end
end
```